HDOC-62

Merge Two Sorted Arrays Using Pointers

C Programming - Day 62 Activity

Problem Statement

Write a C program that reads the sizes and elements of two sorted variable-length integer arrays, arr1[] and arr2[], containing elements in ascending order. Pass both arrays to a function named merge_sorted_arrays() using their addresses. The function must merge the arrays into a third array, preserve ascending order and duplicate elements, display the merged array separated by spaces, and use the two-pointer technique efficiently.

Aim

To implement an efficient C program that merges two sorted integer arrays into a third sorted array using pointers and the two-pointer technique while preserving duplicate elements.

Requirements / Concepts Used

- Variable Length Arrays (VLA)
- Pointers and array addresses
- User-defined function: merge_sorted_arrays()
- Two-pointer merging technique
- Loops and conditional statements
- Standard input/output using <stdio.h>

Algorithm / Logic

1. Input the sizes n1 and n2 of the two arrays.
2. Declare variable-length arrays arr1[n1], arr2[n2], and merged[n1 + n2].
3. Read the elements of arr1 and arr2 in ascending order.
4. Pass the addresses of arr1, arr2, and merged along with their sizes to merge_sorted_arrays().
5. Inside the function, use two indices/pointers i and j, initially 0, for arr1 and arr2. Use k for merged.
6. Compare arr1[i] and arr2[j]. Copy the smaller value to merged[k]. If values are equal, the <= comparison copies one first and the other remains for the next iteration, so duplicates are preserved.
7. When one array is exhausted, copy all remaining elements of the other array into merged.
8. Display all n1 + n2 elements of merged separated by spaces.

C Program

```
#include <stdio.h>

void merge_sorted_arrays(int *arr1, int n1, int *arr2, int n2, int *merged)
{
   int i = 0, j = 0, k = 0;

   while (i < n1 && j < n2)
   {
      if (*(arr1 + i) <= *(arr2 + j))
      {
```

```
            *(merged + k) = *(arr1 + i);
            i++;
        }
        else
        {
            *(merged + k) = *(arr2 + j);
            j++;
        }
        k++;
    }

    while (i < n1)
    {
        *(merged + k) = *(arr1 + i);
        i++;
        k++;
    }

    while (j < n2)
    {
        *(merged + k) = *(arr2 + j);
        j++;
        k++;
    }
}

int main(void)
{
    int n1, n2, i;

    printf("Enter size of first array: ");
    scanf("%d", &n1);
    printf("Enter size of second array: ");
    scanf("%d", &n2);

    if (n1 <= 0 || n2 <= 0)
    {
        printf("Array sizes must be positive.\n");
        return 1;
    }

    int arr1[n1], arr2[n2], merged[n1 + n2];

    printf("Enter %d elements of first sorted array: ", n1);
    for (i = 0; i < n1; i++)
        scanf("%d", &arr1[i]);

    printf("Enter %d elements of second sorted array: ", n2);
    for (i = 0; i < n2; i++)
        scanf("%d", &arr2[i]);

    merge_sorted_arrays(arr1, n1, arr2, n2, merged);

    printf("Merged sorted array: ");
    for (i = 0; i < n1 + n2; i++)
        printf("%d ", merged[i]);
    printf("\n");

    return 0;
```

```
}
```

Program Explanation

- merge_sorted_arrays() receives array addresses through pointer parameters. In C, passing an array name such as arr1 passes the address of its first element.
- The variables i and j act as two pointers/indices that move independently through the two already-sorted input arrays.
- At each step, the smaller current element is copied into merged. Using <= ensures equal values are not discarded, so duplicates are preserved.
- After the main comparison loop ends, the remaining elements from the unfinished array are copied directly because they are already sorted.
- Time complexity is O(n1 + n2), and the merged output array requires O(n1 + n2) additional space.

Sample Output

```
Enter size of first array: 5
Enter size of second array: 4
Enter 5 elements of first sorted array: 1 3 5 7 9
Enter 4 elements of second sorted array: 2 3 6 8
Merged sorted array: 1 2 3 3 5 6 7 8 9
```

Test Cases

Test	arr1[]	arr2[]	Expected merged[]	Observation
1	1 3 5 7 9	2 3 6 8	1 2 3 3 5 6 7 8 9	Normal merge with duplicate 3
2	1 2 3	4 5 6	1 2 3 4 5 6	All first-array elements are smaller
3	4 5 6	1 2 3	1 2 3 4 5 6	All second-array elements are smaller
4	1 2 2 5	2 2 3	1 2 2 2 2 3 5	Multiple duplicates are preserved
5	-5 -1 4	-3 0 4 10	-5 -3 -1 0 4 4 10	Negative values and duplicate 4

Compilation and Execution

Save the source file as hdoc62.c
Compile: gcc hdoc62.c -o hdoc62
Run: ./hdoc62

Result

The program successfully merges two sorted variable-length integer arrays into a third array in ascending order using pointers and the two-pointer technique. Duplicate elements are preserved, and the merged elements are displayed separated by spaces. The algorithm runs in O(n1 + n2) time.